

DOCUMENTO DE MODELAGEM DE SOFTWARE

SISTEMA TECHCITY CONTROL — MÓDULO 2: SISTEMA DE ENTREGAS COM DRONES

RELATÓRIO TÉCNICO – ENTREGA 2: EVOLUÇÃO ARQUITETURAL, GERENCIADORES E REQUISITOS AVANÇADOS

Autores e Corpo de Engenharia:

- **HIAGO DOS SANTOS** (Gestão de Ambiente, Integração e Validação de Sistemas)
 - *Contribuição Técnica:* Responsável pela consolidação do ambiente operacional de testes, implantação de scripts de simulação unificados (`techcity2.js`) e validação de ponta a ponta das regras de integridade do sistema. Garantiu o correto funcionamento das travas lógicas de bateria e manutenção diretamente em console, liderando a homologação técnica da Entrega 2.
- **FABIO JOSÉ DOS SANTOS FILHO** (Desenvolvimento, Refatoração e Regras de Negócio)
 - *Contribuição Técnica:* Responsável pela codificação e programação das regras de negócio do sistema modular. Implementou os algoritmos de controle de estado do ciclo de vida dos drones, a lógica matemática de depreciação e restauração energética da bateria (consumo de 25% por voo e trava de segurança), e as validações de segurança para o fluxo de oficina.
- **GUILHERME WILLIAM DOS SANTOS SILVA** (Arquitetura de Software e Modelagem Fina)
 - *Contribuição Técnica:* Responsável pela concepção e desenho do rearranjo estrutural das coleções. Idealizou a transição do controlador centralizado para a nova topologia baseada em subgerenciadores independentes encapsulados, estabelecendo o mapeamento conceitual através do padrão de projeto *Facade* (Fachada).

1. Descrição Geral da Evolução do Sistema

Na presente etapa (Entrega 2), o Módulo 2 do **TechCity Control** passou por uma reestruturação arquitetural profunda. O antigo controlador centralizado, que acumulava múltiplas responsabilidades de armazenamento e validação, foi desmembrado em uma arquitetura de **Gerenciadores Especializados**, mitigando o acoplamento e promovendo alta coesão estrutural.

A interação externa com o sistema agora é mediada estritamente pelo padrão de projeto **Facade (Fachada)** através da classe **TechCityController**, que atua como único ponto de entrada para a execução das operações municipais em memória. Além da evolução estrutural, foram incorporados novos requisitos operacionais rígidos de segurança pública urbana, governando a autonomia energética (bateria) e o ciclo de vida técnico das aeronaves (manutenção preventiva e corretiva).

2. Especificação dos Requisitos Funcionais Avançados (RF)

Com a evolução do sistema, os seguintes requisitos funcionais foram integrados às regras de negócio:

- **RF11 – Divisão de Responsabilidades por Gerenciadores:** O sistema deve segmentar suas coleções de dados e métodos de manipulação direta em controladores específicos por domínio (Usuários, Drones, Pedidos, Entregas e Auditoria).
- **RF12 – Controle Dinâmico de Autonomia Energética (Bateria):** Cada drone cadastrado deve possuir um indicador de bateria variando de 0% a 100%. Cada decolagem/início de entrega autorizado deve decrementar automaticamente exatamente **25%** da carga atual do VANT.
- **RF13 – Bloqueio de Decolagem por Bateria Crítica:** O sistema deve impedir terminantemente o despacho de qualquer drone cujo nível de bateria seja inferior a **25%**, emitindo um alerta de segurança tratável.
- **RF14 – Comando de Abastecimento Técnico (Recarga):** Operadores municipais ativos devem possuir a capacidade de invocar a rotina de recarga, restaurando o insumo energético de um drone selecionado para 100%.
- **RF15 – Ciclo de Vida de Manutenção Técnico-Operacional:** O sistema deve permitir a alteração do status de um drone para "Manutenção". Drones neste estado ficam impedidos de assumir novas ordens de frete.
- **RF16 – Blindagem Operacional em Voo:** O sistema deve bloquear qualquer tentativa de enviar um drone para a manutenção enquanto o seu status operacional for igual a "Em Operação", evitando inconsistências logísticas em pleno voo.

3. Nova Organização Arquitetural e Atribuição de Responsabilidades

Para sanar o problema de acúmulo de responsabilidades ("God Class"), a lógica de controle foi distribuída nas seguintes estruturas de microserviços internos/controllers:

1. **GerenciadorUsuarios:** Centraliza a coleção de operadores municipais, respondendo pelas validações de existência e de suspensões administrativas (estados ativos/inativos).
2. **GerenciadorDrones:** Detém o inventário das aeronaves públicas e manipula as alterações diretas de integridade e transições de status da frota.
3. **GerenciadorPedidos:** Responsável pela custódia e triagem de demandas e cargas enviadas pela prefeitura.
4. **GerenciadorEntregas:** Responsável pela instanciação das classes de associação e monitoramento do progresso das missões.

5. **ServicoLog**: Gerencia o repositório cronológico imutável de operações e eventos para fins de auditoria governamental.
6. **TechCityController (A Fachada)**: Consolida instâncias de todos os subgerenciadores descritos acima. Esta classe não expõe as coleções internas e fornece apenas assinaturas limpas de métodos de alto nível para os scripts de simulação.

4. Modelagem Conceitual do Fluxo de Estados (Máquina de Estados do Drone)

A propriedade `#status` da entidade `Drone` agora responde a uma máquina de estados robusta, cujas transições são rigidamente coordenadas e auditadas pelo sistema

- **Regra de Transição 1:** `Disponível` → `Em Operação` (Requer: Operador Ativo + Carga compatível + Bateria ≥ 25%).
- **Regra de Transição 2:** `Disponível` → `Manutenção` (Autorizado a qualquer momento por operador ativo).
- **Regra de Transição 3:** `Em Operação` → `Manutenção` (**Bloqueio Absoluto**; o drone precisa concluir a entrega e retornar ao estado Disponível primeiro).

5. Considerações de Encapsulamento de Dados e Modificadores Privados

Em estrito cumprimento aos padrões clássicos da Programação Orientada a Objetos (POO), todas as novas propriedades concebidas nesta etapa foram blindadas sob escopo privado imutável por meio da sintaxe nativa `#` do JavaScript.

Os subgerenciadores internos são instanciados como propriedades privadas dentro do `TechCityController` (ex: `this.#gerenciadorDrones`), impedindo que scripts externos acessem ou modifiquem os arrays de dados diretamente sem passar pelas regras de validação da Fachada. As mudanças de estado energético (`#bateria`) e operacionais (`#status`) são mediadas por métodos semânticos expressivos, tais como `consumirBateria(qtd)` e `recarregar()`, garantindo o isolamento completo das entidades de negócio.

6. Roteiro de Validação e Evidência de Sucesso de Execução

A simulação automatizada do sistema executada via terminal demonstrou o funcionamento perfeito da modelagem adotada, conforme evidenciado no log de execução real coletado:

1. **Cenário de Degradação de Energia:** O sistema realizou com sucesso quatro decolagens sequenciais do `VANT-ALFA`. O decréscimo acumulado reduziu a bateria de 100% para 0% (25% por voo). Na quinta tentativa de voo, o sistema disparou a trava do `RF13`, abortando a missão por insuficiência energética.
2. **Cenário de Restrição Técnico (Oficina):** O drone `VANT-BETA` foi transferido para o status de "Manutenção". Uma ordem de despacho subsequente tentou alocar esta

aeronave e foi imediatamente rechaçada pelo sistema, comprovando a eficácia do isolamento de segurança.

3. **Auditoria Geral:** O log cronológico registrou de forma detalhada e sequencial as assinaturas digitais dos operadores em cada um dos eventos bem-sucedidos do sistema, atendendo plenamente aos quesitos de rastreabilidade do projeto.